

Инструкция к работе над фронтендом итогового проекта

Время чтения — 4 минуты

Содержание:

1. Цель работы
2. Этапы выполнения проекта
 - Создание проекта
 - Настройка проекта
 - Разработка интерфейса
 - Работа с API
 - Маршрутизация
 - Добавление Service Worker
 - Сборка проекта
3. Чек-лист выполнения проекта
4. Результаты работы над проектом

Цель работы

Создать веб-приложение с использованием HTML, CSS, JavaScript и React.

В проекте должны быть реализованы:

- UI-компоненты, разработанные ранее на курсе;
- ключевые функции: маршрутизация, авторизация, работа с API и использование Service Worker.

Этапы выполнения проекта

1. Создание проекта

1. Установите [Node.js](#) (версия 18 или выше).
2. Создайте новую папку для проекта:

```
mkdir my-frontend-project && cd my-frontend-project
```

3. Инициализируйте проект с помощью npm:

```
npm init -y
```

4. Установите инструменты сборки (например, Vite):

```
npm create vite@latest . --template react
```

Следуйте инструкциям и выберите шаблон для React.

2. Настройка проекта

1. Установите дополнительные зависимости:

```
npm install axios react-router-dom
```

Эти библиотеки понадобятся для работы с API и маршрутизации.

2. Настройте ESLint и Prettier:

- Установите пакеты:

```
npm install eslint prettier eslint-plugin-react eslint-config-prettier --save-dev
```

- Создайте файлы конфигурации `.eslintrc.json` и `.prettierrc`.

3. Разработка интерфейса

1. Создайте базовую структуру проекта:

- `src/components` — для UI-компонентов;
- `src/pages` — для страниц;
- `src/styles` — для стилей;
- `src/App.jsx` — основной файл маршрутизации.

2. Перенесите UI-компоненты из предыдущих заданий в `src/components`.

3. Разработайте базовые страницы приложения:

- Главная страница (Home) — информация о приложении.
- Авторизация (Login) — вход пользователя.
- Каталог (Products) — отображение списка товаров (если эта страница необходима).

4. Работа с API

1. Настройте файл src/api.js:

```
import axios from 'axios';
const api = axios.create({
  baseURL: 'https://api.example.com',
});
export default api;
```

2. Реализуйте запросы к API, например:

- авторизация пользователя;
- загрузка списка товаров.

5. Маршрутизация

1. Настройте маршруты в App.jsx, используя react-router-dom:

```
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';
import Home from './pages/Home';
import Products from './pages/Products';
import Login from './pages/Login';

function App() {
  return (
    <Router>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/products" element={<Products />} />
        <Route path="/login" element={<Login />} />
      </Routes>
    </Router>
  );
}

export default App;
```

6. Добавление Service Worker

1. Установите библиотеку workbox:

```
npm install workbox-webpack-plugin
```

2. Настройте Service Worker для офлайн-режима и кэширования данных.

7. Сборка и развёртывание

1. Выполните команду сборки run build:

```
npm run build
```

2. Проверьте папку dist для подтверждения успешной сборки.

3. Разместите проект на платформе:

- [Vercel](#);
- [Netlify](#);
- сервер с Docker и Nginx.

Чек-лист выполнения проекта

Структура проекта	Все страницы находятся в src/pages
	UI-компоненты лежат в src/components
	Общий файл стилей в src/styles
Функциональность	Работает авторизация пользователя
	Данные загружаются из API
	Работает маршрутизация между страницами
Сборка и развёртывание	Проект собирается без ошибок
	Проект доступен по публичной ссылке

Результаты работы над проектом

К концу работы над проектом у вас будет готовое веб-приложение с:

- привлекательным UI;
- реализованной маршрутизацией и работой с API;
- авторизацией;
- оптимизированной сборкой и кэшированием данных.